



EE 463

Hardware Project - Final Report

Group Name: Buck'ı Yedik

Members:

Ece Canbaz

Maide Esra Şirin

Battal Emre Sevinç

Date: 18.01.2026

Contents

1	Introduction	3
2	Design Decisions and Topology Selection	3
2.1	Considered Topologies	3
2.1.1	Single Phase Rectifier with Buck Converter	3
2.1.2	Three-Phase Rectifier with Buck Converter	4
2.1.3	Full Bridge DC-DC Converter	4
2.2	Selected Topology	4
3	Simulations	6
3.1	Simulation Parameters and Setup	6
3.2	No-Load Simulation Results	6
3.3	Full Load Simulation Results	8
3.3.1	Soft Start Performance	8
3.3.2	Steady State Ripple Analysis (Full Load)	8
3.3.3	Rectifier Stage Output Voltage (V_{rec})	9
3.3.4	Output Voltage Analysis (V_{out})	9
3.3.5	Verification of Gate Signal Evolution	10
3.4	Discussion: No-Load vs Full Load Analysis	11
4	Component Selection	12
4.1	Three-Phase Rectifier Module	12
4.2	DC Link Capacitor	12
4.3	Power MOSFET	13
4.4	Freewheeling Diode	13
4.5	Gate Driver	14
4.6	Microcontroller	14
4.7	Fuse	14
4.8	15 V to 5 V DC-DC Voltage Regulator	14
4.9	Cooling Fan	15
5	Loss and Thermal Analysis	15
5.1	Three-Phase Rectifier Module	16
5.1.1	Conduction Loss	16
5.1.2	Thermal Estimation with Heatsink	16
5.2	Power MOSFET	16
5.2.1	Conduction Loss	16
5.2.2	Switching Loss	16
5.2.3	Thermal Estimation with Heatsink and Fan	17
5.3	Freewheeling Diode	17
5.3.1	Conduction Loss	17
5.3.2	Reverse-Recovery Loss	17
5.3.3	Thermal Estimation with Heatsink and Fan	18
5.4	Total Semiconductor Loss and Efficiency	18
5.5	Discussion of Results	18

6	Implementation and Test Results	18
6.1	Implementation Process	18
6.2	Demonstration Process	24
7	Conclusion	26
8	References	26
9	Appendix	28
9.1	Arduino Code	28

1 Introduction

In this project report, a High Voltage DC Motor Driver is designed and implemented. The main goal is to convert three-phase AC grid voltage into a controllable DC output up to 180 V to run a separately excited DC motor.

The project includes theoretical analysis, simulations, and hardware testing. The design is verified using PSIM and LTspice software. The final hardware implementation uses a Low-Side Switching Buck Converter topology controlled by an Arduino. This digital approach is chosen to provide safer operation and better control compared to the initial analog high-side design.

In this report, Section 2 explains the design choices and topologies considered. Section 3 shows the simulation results. Sections 4 and 5 cover component selection and thermal analysis. Finally, Section 6 describes the physical setup and the test results.

2 Design Decisions and Topology Selection

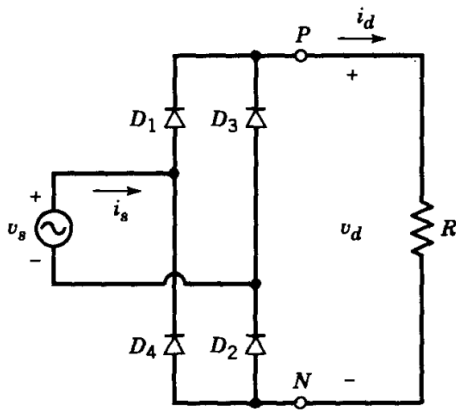
In this section, possible topologies for the DC Motor Drive are discussed. The main goal is to select a topology that provides a stable output voltage, high efficiency, and manageable control complexity for a 1.6 kW system. Three main topologies were considered.

2.1 Considered Topologies

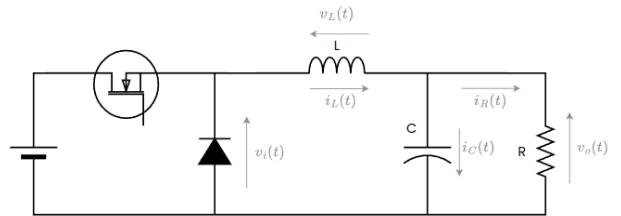
2.1.1 Single Phase Rectifier with Buck Converter

This topology consists of a single-phase diode bridge rectifier followed by a DC-DC Buck converter.

- **Advantages:** It is the simplest and most cost-effective solution with fewer components.
- **Disadvantages:** It draws high current from a single phase, causing grid unbalance. Furthermore, the ripple in the DC bus voltage is very high, so we need very large capacitor banks to provide a stable voltage for the motor.



(a) Single-Phase Rectifier Stage



(b) Buck Converter Stage

Figure 1: Single-Phase Rectifier and Buck Converter

2.1.2 Three-Phase Rectifier with Buck Converter

This system uses a three-phase diode bridge to rectify the AC voltage and a buck converter to regulate the voltage.

- **Advantages:** The three-phase rectification provides a much smoother DC voltage with less ripple compared to the single-phase option. It draws balanced power from the grid.
- **Disadvantages:** It requires more diodes than a single-phase rectifier, leading to higher losses, but this is acceptable considering the resulting power quality.

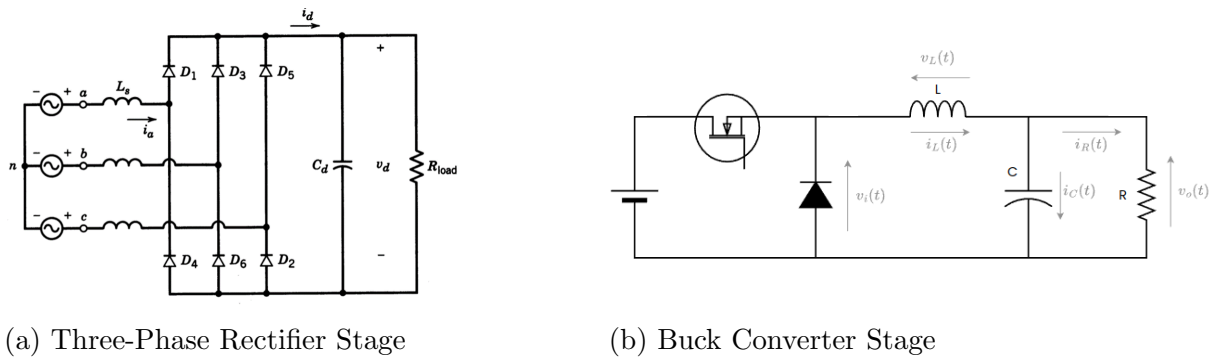


Figure 2: Three-Phase Rectifier and Buck Converter

2.1.3 Full Bridge DC-DC Converter

This topology uses four switches in an H-Bridge configuration.

- **Advantages:** It allows four-quadrant operation, meaning it can drive the motor in both directions and supports regenerative braking.
- **Disadvantages:** It requires four switching devices and a complex gate drive circuit. Since bidirectional motion was not a primary requirement for this project, the added complexity and cost were seemed unnecessary.

2.2 Selected Topology

After evaluating the options, the Three-Phase Rectifier with Buck Converter was selected. This topology offers the best balance between performance and complexity for a 1.6 kW load.

For the implementation, a strategic decision was made to use a Low-Side Switching technique based on the recommendations we received during the project feedback session. In standard high-side buck converters, driving the gate of the switch requires isolated or bootstrap circuitry, which can be complex. By placing the switch on the low side, the source of the MOSFET is connected to the ground. This allows the Arduino to drive the switch more easily and safely without complex isolation, simplifying the prototyping process.

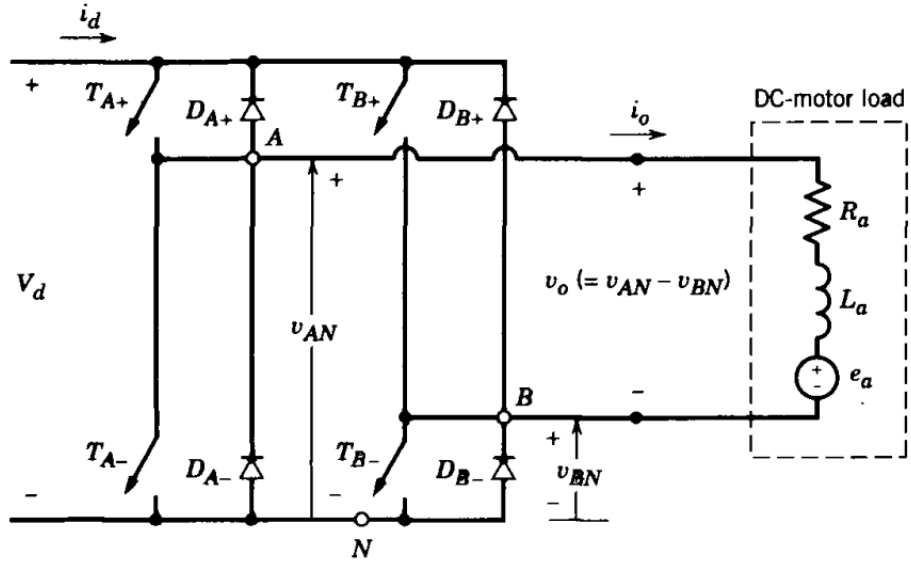


Figure 3: Full Bridge DC-DC Converter Topology

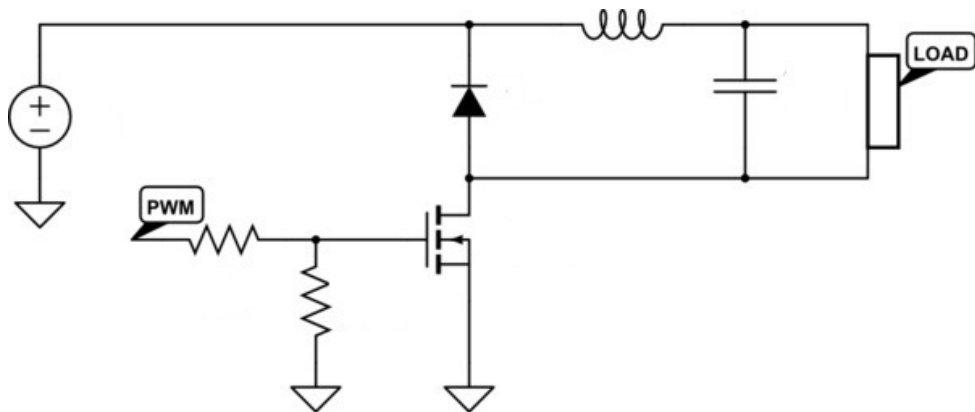


Figure 4: Low-Side Switching Buck Converter

3 Simulations

In this section, the performance of the designed DC Motor Drive is verified using PSIM software. The simulations are conducted under two main scenarios: No-Load and Full Load conditions. The motor parameters used in the simulation are updated based on the actual nameplate data of the Crompton Parkinson DC Motor.

3.1 Simulation Parameters and Setup

The simulation circuit model includes the three-phase rectifier, the buck converter stage, and the DC machine model. To reflect the real-world behaviour of the motor, the armature resistance (R_a) and inductance (L_a) were adjusted to include the interpole windings, and the rated current was set to 23.4 A as specified on the motor nameplate. We control the MOSFET gate signal with PSIM C block code using the same code that we used in Arduino. 2 DC machines is used, one as a motor and other one as a generator. A resistive load is connected to generator side to simulate the kettle bonus.

The overall simulation schematic is shown in Figure 5.

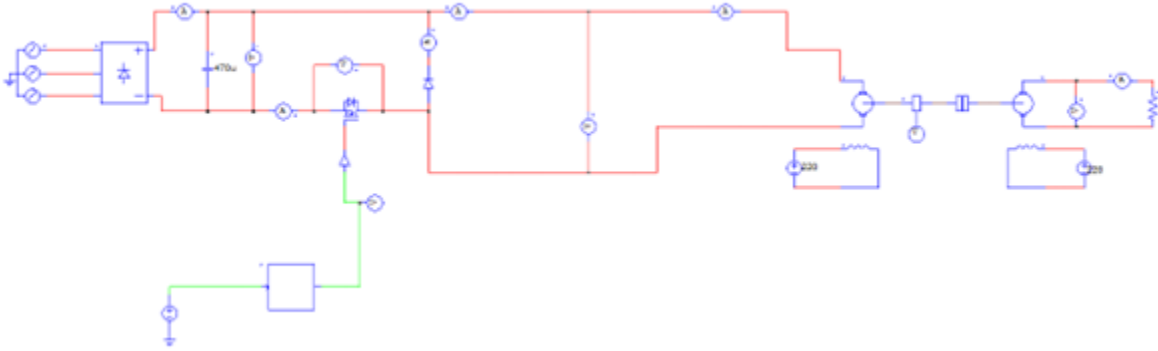


Figure 5: PSIM Simulation Schematic of the DC Motor Drive.

3.2 No-Load Simulation Results

The simulation analysis begins with the No-Load condition. In a real-world scenario, even when unloaded, the DC motor experiences mechanical friction (bearings) and windage losses. To model this accurately in PSIM, a constant torque load of approximately 2 Nm was applied to the motor shaft in order to reach the current value in the test.

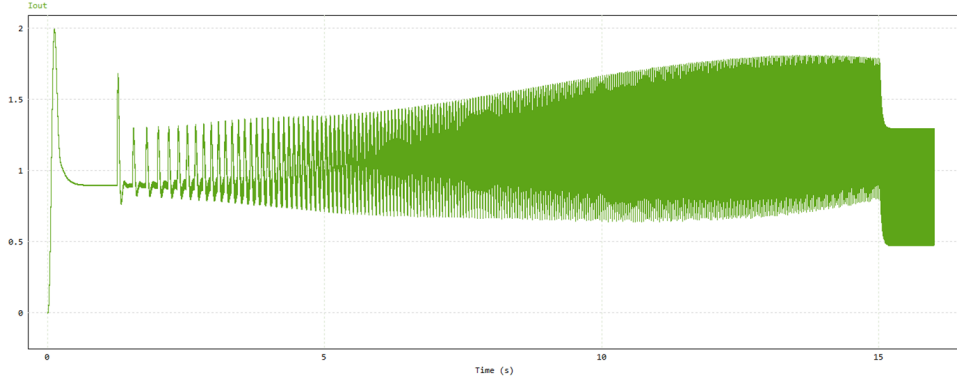


Figure 6: Armature Current under No-Load Condition (with Friction Load).

As observed in Figure 6, the motor draws a continuous current in the range of 1.3 A – 1.8 A to overcome the friction load. This aligns with the experimental measurements where the motor drew approximately 2 A at no-load. Figure 7 details the current ripple in this steady state.

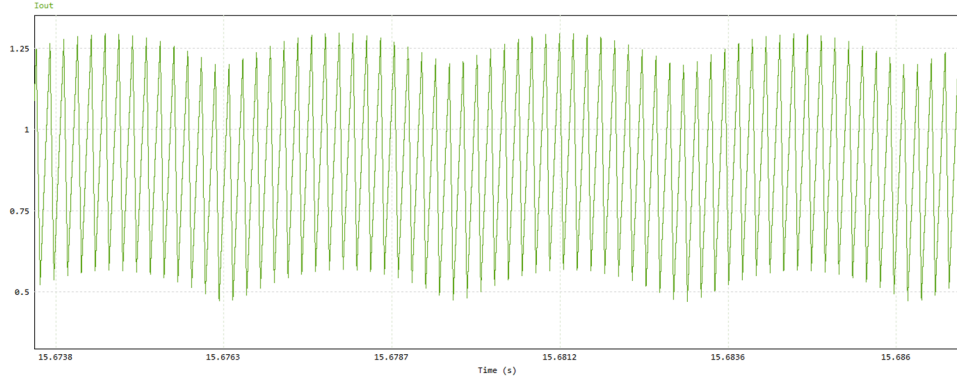


Figure 7: Current Ripple Waveform under No-Load Condition.

Finally, the output voltage regulation under no-load condition is presented in Figure 8. The system successfully maintains the target voltage level (approx. 180 V) with stability, demonstrating that the buck converter controller functions correctly even at light loads.

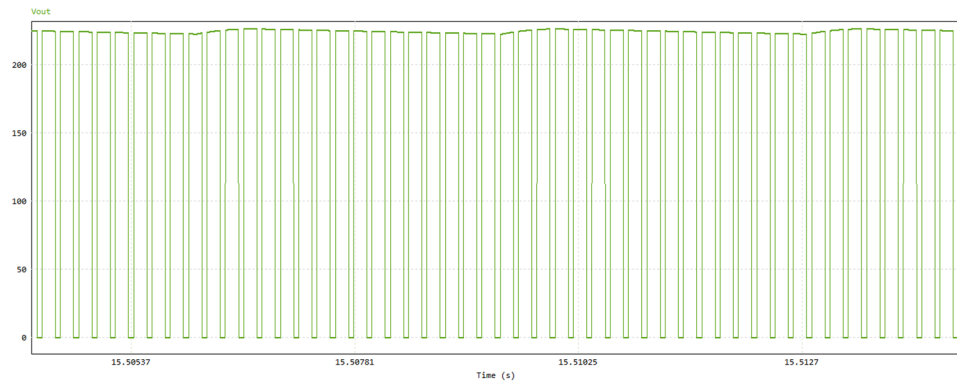


Figure 8: Output Voltage (V_{out}) under No-Load Condition.

3.3 Full Load Simulation Results

Following the no-load verification, the system was tested under Full Load conditions.

3.3.1 Soft Start Performance

To protect the system from high inrush currents, a soft-start algorithm was implemented. Figure 9 demonstrates the effectiveness of this approach; the armature current rises gradually over the first 15 seconds and stays within safe limits.

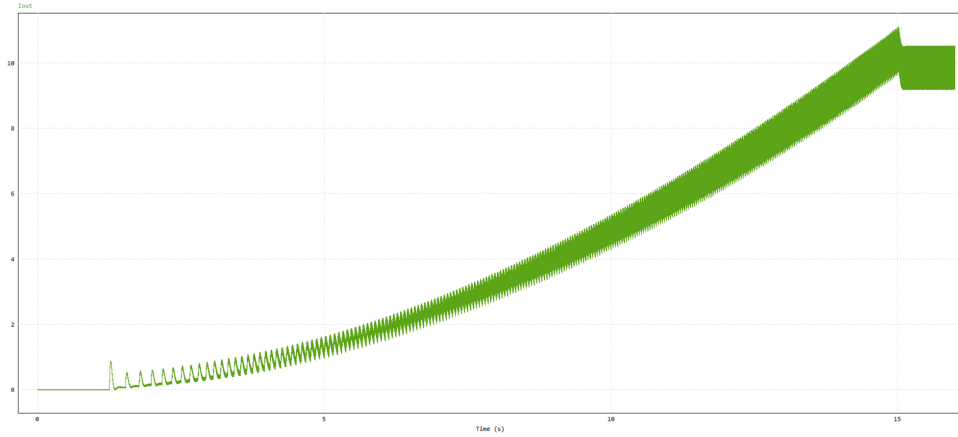


Figure 9: Armature Current (I_{out}) during Soft Start (Full Load).

Simultaneously, the motor speed ramps up smoothly to the rated speed without overshoot (Figure 10).

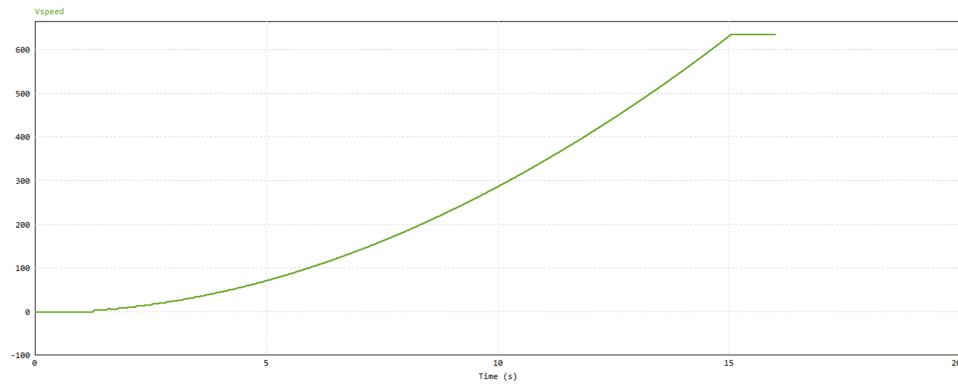


Figure 10: Motor Speed Response during Start-up.

3.3.2 Steady State Ripple Analysis (Full Load)

Under full load, the buck converter operates in Continuous Conduction Mode (CCM).

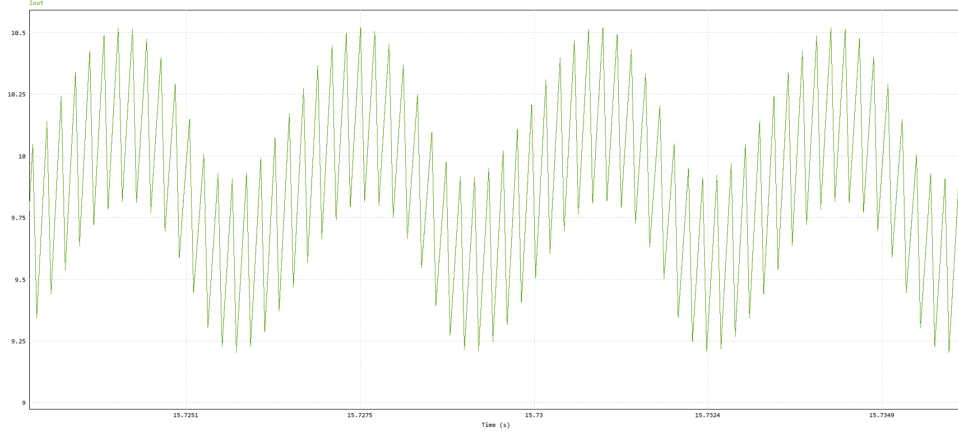


Figure 11: Steady State Output Current Ripple at Full Load.

3.3.3 Rectifier Stage Output Voltage (V_{rec})

The input to the buck converter is provided by the three-phase diode bridge rectifier. Figure 12 shows the DC link voltage (V_{rec}), which stabilizes around 250 V.

Since the supply is a three-phase 50 Hz AC grid, the rectified voltage exhibits a characteristic 300 Hz ($6 \times f_{grid}$) ripple. The selected 470 μF DC-link capacitor effectively smooths this voltage, keeping the ripple within acceptable limits to ensure a stable supply for the switching element.

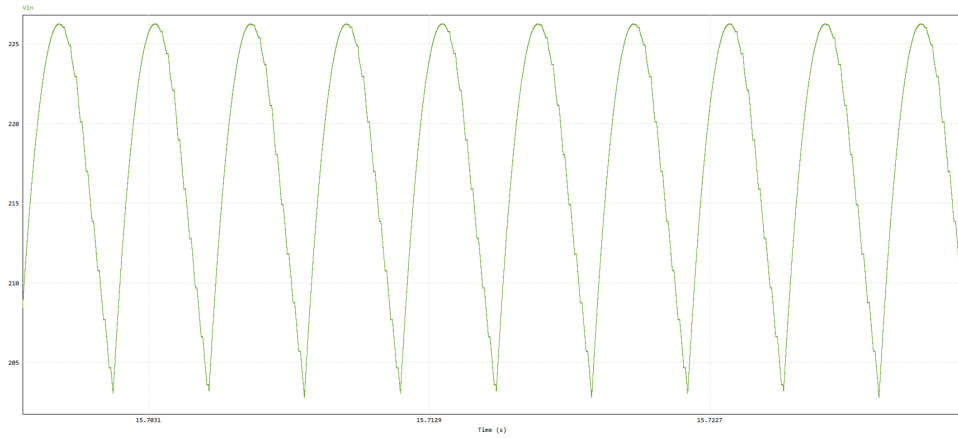


Figure 12: Rectifier Output Voltage (V_{rec}) showing 300 Hz Ripple.

3.3.4 Output Voltage Analysis (V_{out})

The output voltage regulation capability of the drive was verified under full load. As illustrated in Figure 13, the buck converter successfully maintains the average output voltage at the target level of approximately 180 V after the soft-start period.

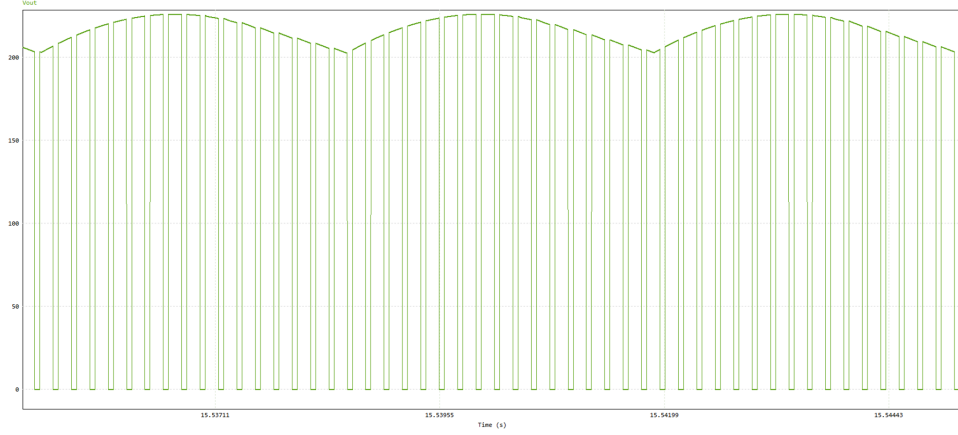


Figure 13: Steady State Output Voltage (V_{out}) under Full Load (180 V).

3.3.5 Verification of Gate Signal Evolution

To further verify the soft-start implementation, the MOSFET gate signals (PWM duty cycle) were analyzed at two different timestamps: $t = 5s$ (during the ramp-up) and $t = 15s$ (steady state).

Figure 14 shows the gate signal at $t = 5s$. Since the soft-start algorithm uses a quadratic ramp function ($Duty \propto t^2$), the effective duty cycle at this stage is significantly low.

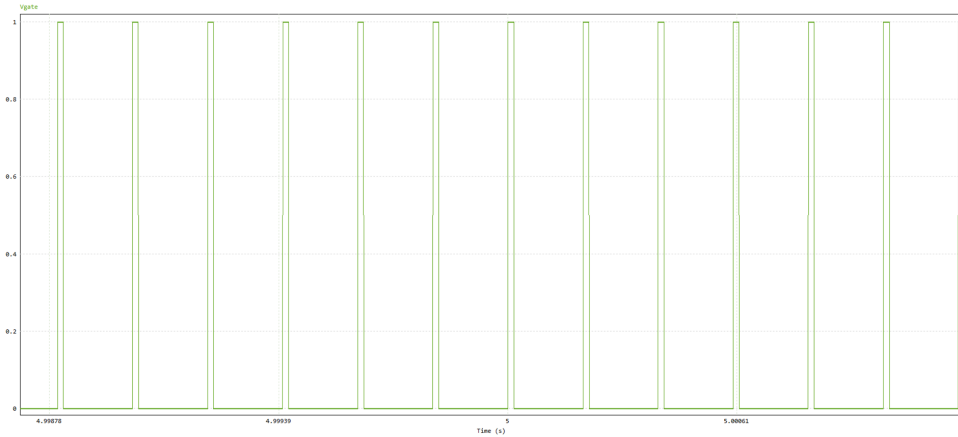


Figure 14: MOSFET Gate Signal at $t = 5s$ (Narrow Duty Cycle).

In contrast, Figure 15 shows the gate signal at $t = 15s$, where the soft-start period ends. Here, the duty cycle has reached its maximum set value ($D_{max} = 0.72$), resulting in much wider pulses.

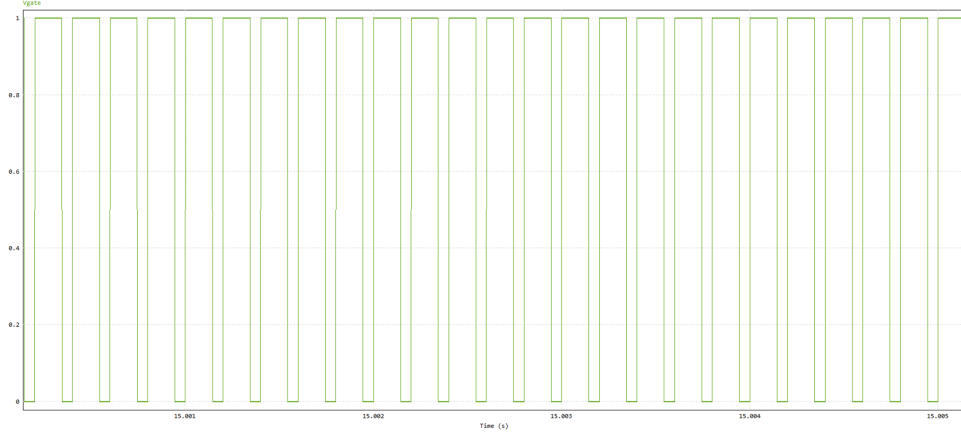


Figure 15: MOSFET Gate Signal at $t = 15s$ (Maximum Duty Cycle).

Comparison: The visual difference between the narrow pulses at $5s$ and the wide pulses at $15s$ confirms that the Arduino controller correctly modulates the PWM width over time, ensuring a smooth voltage build-up at the output.

3.4 Discussion: No-Load vs Full Load Analysis

The comparison between No-Load and Full Load simulation scenarios provides critical insights into the drive's static and dynamic performance characteristics:

- **Load Dependent Current Draw:** The simulation results clearly validate the motor model accuracy. In the No-Load scenario, the armature current stabilizes around 1.5 A (Figure 6), which corresponds to the torque required to overcome mechanical friction and windage losses. In contrast, under Full Load, the current rises to the rated level to support the external load. The transition from 1.5 A to the rated current demonstrates the power stage's capability to handle the full dynamic range of the motor.
- **Ripple Characteristics and Conduction Modes:** A key observation is the behaviour of the current ripple (ΔI). Since the inductance of the motor, switching frequency (5 kHz), and input voltage are relatively constant, the *absolute magnitude* of the ripple remains similar in both cases. However, the *percentage ripple* ($\frac{\Delta I}{I_{avg}}$) is significantly higher in the No-Load condition due to the low average current.

Crucially, the simulation confirms that the inherent friction of the motor keeps the current continuous enough to prevent the system from entering deep Discontinuous Conduction Mode (DCM), which could otherwise degrade voltage regulation.

- **Voltage Regulation Stability:** Regardless of the drastic difference in load current (from $\approx 1.5\text{ A}$ to rated current), the output voltage (V_{out}) settled at the target 180 V in both scenarios (Figure 8 and Figure 13).
- **Transient Response and Safety:** The Full Load scenario exhibits the potential for massive inrush currents due to the combined effect of mechanical inertia and load torque. The 15 second ramp implemented in the simulation effectively mitigates this risk, protecting the motor, semiconductor switches and the fuse from thermal and current stress.

Overall, the simulations validate the design decisions, confirming that the buck converter topology with the selected components can reliably drive the DC motor from standstill to full load.

4 Component Selection

This section details the critical components selected for the DC Motor Drive. All semiconductor devices were chosen with voltage and current ratings designed to handle the operational stresses and provide safety margins.

4.1 Three-Phase Rectifier Module

For the three-phase rectifier stage, the SKBPC5016 bridge module is selected. It is a three-phase bridge rectifier rated for an average output current of 50 A and a repetitive peak reverse voltage of 1600 V. The SKBPC5016 also has a high surge current capability, which is important to handle the inrush current when the large DC-link capacitor is first charged. SKBPC5016 can be seen in Figure 16



Figure 16: Selected three-phase full bridge rectifier

4.2 DC Link Capacitor

The KEMET ALA7DA471DC400 ($470 \mu F$, 400 V) was selected. Since the system input was reduced to $185 V_{AC}$, the resulting DC bus is approximately 260 V. Therefore, the 400 V rating provides a sufficient safety margin without requiring a series connection, while the $470 \mu F$ capacitance effectively smooths the voltage ripple.



Figure 17: Selected KEMET ALA7DA471DC400 Capacitor

4.3 Power MOSFET

The STW20NM60 N-Channel MOSFET was selected for this project. Initially, the design was planned to operate with the full grid voltage, resulting in a DC-link voltage of approximately 560 V. Therefore, a MOSFET with a 600 V drain-to-source voltage rating was a suitable choice.

However, during the implementation phase, we decided to reduce the input voltage, resulting in a DC-link voltage of approximately 250 V. Since the component was already selected and purchased before this decision, we continued to use the STW20NM60. Using a 600 V MOSFET for a 250 V application is not ideal and slightly increased our conduction losses compared to a lower-voltage switch. However, it was kept due to availability.

Despite this, the device performs well. Its 20 A current capability is sufficient to safely handle both the normal motor current and high starting currents. The TO-247 package allows effective heat dissipation using a heatsink.



Figure 18: Selected Power MOSFET

4.4 Freewheeling Diode

As the freewheeling diode, the DSI30-06A (DSEI30-06A) was selected since its 600 V reverse voltage rating is compatible with the DC-link level of the system and its 30 A current rating provides sufficient margin for the expected operating conditions. It is an ultrafast/soft-recovery diode, which helps to reduce reverse-recovery related losses and switching stress in the buck converter.

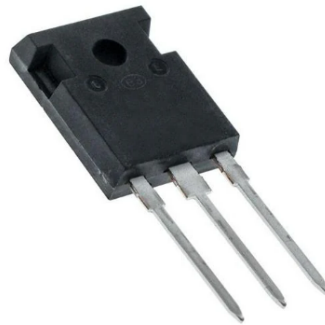


Figure 19: Selected Diode

4.5 Gate Driver

To interface the digital control logic with the high-power switching stage, the FOD3120 gate driver is used. The primary reason for selecting this component is its ability to provide galvanic isolation, effectively separating the low-voltage Arduino circuit from the high-voltage power lines. In addition to safety, the FOD3120 offers a high output current capability of 2.5 A, which allows it to charge and discharge the MOSFET's gate capacitance rapidly, ensuring clean and fast switching transitions.

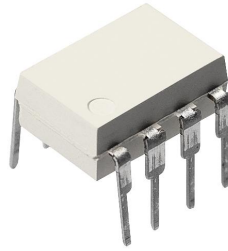


Figure 20: Selected Gate Driver

4.6 Microcontroller

The Arduino Uno was selected as the digital controller for the system. The primary reason for this choice is its ability to generate PWM signals with great ease and precision. Unlike complex analog control circuits that require tuning external components, the Arduino allows for straightforward generation and adjustment of the PWM duty cycle via software. In our implementation, the PWM duty cycle is controlled in real time using a potentiometer connected to an analog input of the Arduino. This significantly simplified the implementation of the control logic. The complete source code developed for this project is provided in the Appendix.

4.7 Fuse

We placed a 15A fuse at the output of the DC link capacitor to protect the circuit from high current.

4.8 15 V to 5 V DC-DC Voltage Regulator

In our design, the gate driver requires a 15 V supply, while the Arduino operates at 5 V. Since we used a single power supply for the whole system, we added a MP1584 module to step down 15 V to a regulated 5 V for the Arduino. This allowed us to power both the gate driver and the controller from one supply rail.



Figure 21: Selected Voltage Regulator

4.9 Cooling Fan

To reduce the temperature rise of the main power devices, a small cooling fan (DA04010S12H, 12 V, 0.10 A) was added and positioned to blow air directly over the MOSFET and the freewheeling diode, since these components contribute significantly to the total losses. Because the available auxiliary supply is 15 V while the fan is rated for 12 V, a 10 Ω stone resistor was connected in series with the fan to drop the voltage.



Figure 22: Selected cooling fan

5 Loss and Thermal Analysis

The worst-case thermal and loss calculations are carried out at the maximum output voltage $V_o = 180$ V and rated power $P_o = 1.6$ kW. The DC-link voltage is $V_{dc} \approx 250$ V and the switching frequency is $f_s = 5$ kHz. Thus,

$$D = \frac{V_o}{V_{dc}} = \frac{180}{250} = 0.72, \quad I_o = \frac{P_o}{V_o} = \frac{1600}{180} = 8.89 \text{ A}, \quad I_{dc} \approx \frac{P_o}{V_{dc}} = \frac{1600}{250} = 6.40 \text{ A}. \quad (1)$$

Ambient temperature is taken as $T_{amb} = 30^\circ\text{C}$.

For thermal calculations, the junction temperature is estimated by

$$T_j \approx T_{amb} + P_{loss} (R_{\theta JC} + R_{\theta CS} + R_{\theta SA}), \quad (2)$$

where $R_{\theta JC}$ is junction-to-case, $R_{\theta CS}$ is the interface (thermal paste + mounting), and $R_{\theta SA}$ is heatsink-to-ambient thermal resistance.

5.1 Three-Phase Rectifier Module

5.1.1 Conduction Loss

In a three-phase diode bridge, the DC current flows through two diodes in series at any time. Therefore, a first-order rectifier conduction loss is

$$P_{BR,cond} \approx 2V_F I_{dc}. \quad (3)$$

Using a conservative $V_F \approx 1.2 \text{ V}$,

$$P_{BR,cond} \approx 2(1.2)(6.40) = 15.36 \text{ W}. \quad (4)$$

5.1.2 Thermal Estimation with Heatsink

The rectifier is mounted on an aluminum TO3/TO66/SOT-type heatsink which is rated around $R_{\theta SA} \approx 6^\circ\text{C/W}$. Using $R_{\theta JC} = 0.9^\circ\text{C/W}$ and a interface resistance $R_{\theta CS} \approx 0.5^\circ\text{C/W}$:

$$T_{j,BR} \approx 30 + (15.36)(0.9 + 0.5 + 6) = 30 + 15.36(7.4) \approx 144^\circ\text{C}. \quad (5)$$

This value is close to the maximum junction rating; therefore, firm mounting, thermal paste, and sufficient airflow around the rectifier heatsink are important.

5.2 Power MOSFET

5.2.1 Conduction Loss

The MOSFET conduction loss is approximated by

$$P_{M,cond} \approx I_o^2 R_{DS(on)} D. \quad (6)$$

Using $R_{DS(on)} = 0.29 \Omega$,

$$P_{M,cond} \approx (8.89)^2(0.29)(0.72) \approx 16.50 \text{ W}. \quad (7)$$

5.2.2 Switching Loss

A standard hard-switching approximation gives

$$P_{M,sw} \approx \frac{1}{2} V_{dc} I_o (t_r + t_f) f_s. \quad (8)$$

With $t_r \approx 20 \text{ ns}$ and $t_f \approx 11 \text{ ns}$,

$$P_{M,sw} \approx \frac{1}{2}(250)(8.89)(31 \times 10^{-9})(5000) \approx 0.17 \text{ W}. \quad (9)$$

Hence,

$$P_M \approx P_{M,cond} + P_{M,sw} \approx 16.67 \text{ W}. \quad (10)$$

5.2.3 Thermal Estimation with Heatsink and Fan

The MOSFET is mounted on the 59-AS Component Heatsink and is actively cooled by the fan. Instead of assuming an exact $R_{\theta SA}$ under forced airflow, we first compute the required $R_{\theta SA}$ to keep $T_j \leq 150^\circ\text{C}$:

$$R_{\theta SA, req} \leq \frac{T_{j, max} - T_{amb}}{P_M} - (R_{\theta JC} + R_{\theta CS}). \quad (11)$$

Using $T_{j, max} = 150^\circ\text{C}$, $T_{amb} = 30^\circ\text{C}$, $P_M = 16.67\text{ W}$, $R_{\theta JC} = 0.65^\circ\text{C/W}$, and $R_{\theta CS} \approx 0.5^\circ\text{C/W}$:

$$R_{\theta SA, req} \leq \frac{150 - 30}{16.67} - (0.65 + 0.5) \approx 6.05^\circ\text{C/W}. \quad (12)$$

With the fan blowing directly onto the heatsink, an effective value of $R_{\theta SA} \approx 5^\circ\text{C/W}$ is a reasonable approximation. Thus, the heatsink temperature rise is

$$T_{HS} \approx T_{amb} + P_M R_{\theta SA} \approx 30 + (16.67)(5) \approx 113^\circ\text{C}. \quad (13)$$

The junction is higher than the heatsink by

$$\Delta T_{J-HS} \approx P_M (R_{\theta JC} + R_{\theta CS}) \approx (16.67)(0.65 + 0.5) \approx 19^\circ\text{C}. \quad (14)$$

Therefore, the estimated MOSFET junction temperature is

$$T_j \approx T_{HS} + \Delta T_{J-HS} \approx 113 + 19 \approx 132^\circ\text{C}. \quad (15)$$

This estimate remains below the 150°C junction limit and supports the necessity of forced airflow.

5.3 Freewheeling Diode

5.3.1 Conduction Loss

During the MOSFET off-time, the freewheeling diode conducts the load current. Using the diode's forward model $V_F \approx V_{T0} + r_F I$,

$$P_{D, cond} \approx (V_{T0} + r_F I_o) I_o (1 - D). \quad (16)$$

With $V_{T0} = 1.10\text{ V}$, $r_F = 8.5\text{ m}\Omega$, $I_o = 8.89\text{ A}$, and $1 - D = 0.28$:

$$V_F \approx 1.10 + (0.0085)(8.89) \approx 1.176\text{ V}, \quad P_{D, cond} \approx (1.176)(8.89)(0.28) \approx 2.93\text{ W}. \quad (17)$$

5.3.2 Reverse-Recovery Loss

A conservative reverse-recovery loss estimate is

$$P_{D, rr} \approx \frac{1}{2} V_{dc} I_o t_{rr} f_s. \quad (18)$$

Using $t_{rr} \approx 80\text{ ns}$,

$$P_{D, rr} \approx \frac{1}{2} (250)(8.89)(80 \times 10^{-9})(5000) \approx 0.44\text{ W}. \quad (19)$$

Therefore,

$$P_D \approx P_{D, cond} + P_{D, rr} \approx 3.37\text{ W}. \quad (20)$$

5.3.3 Thermal Estimation with Heatsink and Fan

The diode is mounted on a TO-220 heatsink and is placed under the fan airflow. Using $R_{\theta JC} = 0.8^\circ\text{C/W}$ and $R_{\theta CH} = 0.25^\circ\text{C/W}$, and a representative TO-220 heatsink value $R_{\theta SA} \approx 14^\circ\text{C/W}$:

$$T_{j,D} \approx 30 + (3.37)(0.8 + 0.25 + 14) = 30 + 3.37(15.05) \approx 81^\circ\text{C}. \quad (21)$$

Since the fan further decreases the effective $R_{\theta SA}$, the real junction temperature is expected to be lower.

5.4 Total Semiconductor Loss and Efficiency

The dominant semiconductor losses at the rated point are summarized as

$$P_{\text{loss},\text{total}} \approx P_{BR,\text{cond}} + P_M + P_D \approx 15.36 + 16.67 + 3.37 \approx 35.4 \text{ W}. \quad (22)$$

This corresponds to a first-order efficiency estimate

$$\eta \approx \frac{P_o}{P_o + P_{\text{loss},\text{total}}} = \frac{1600}{1600 + 35.4} \approx 97.8\%. \quad (23)$$

5.5 Discussion of Results

The calculations indicate that the dominant losses are due to MOSFET and rectifier conduction, while switching losses are negligible at $f_s = 5 \text{ kHz}$. Hence, the overall thermal behavior is mainly determined by the conduction path and the heatsink-to-ambient thermal resistance.

The obtained junction temperature values are conservative because worst-case datasheet parameters were used (e.g., $R_{DS(on),\text{max}}$ and simplified diode forward-drop models), and the thermal interface/heatsink conditions introduce uncertainty in $R_{\theta CS}$ and $R_{\theta SA}$. Therefore, the actual temperatures can be lower than the estimated values without contradicting the analysis.

6 Implementation and Test Results

6.1 Implementation Process

In this section, we are going to talk about our implementation process, the test results of the tests conducted during the implementation process, and the final demonstration test results. Also, we would like to mention the problems we encountered throughout the project. First, after we got our components, we tested arduino and gate driver, without mosfet, we observed whether we could achieve soft start or not, and get the necessary PWM signal. After this is completed, we ran the same test, including the MOSFET on a breadboard. You can observe our setup down below:

We successfully observed soft start, our PWM signal reached its final waveform after 3 seconds, as we arranged. You can observe the initial waveform compared to the final:

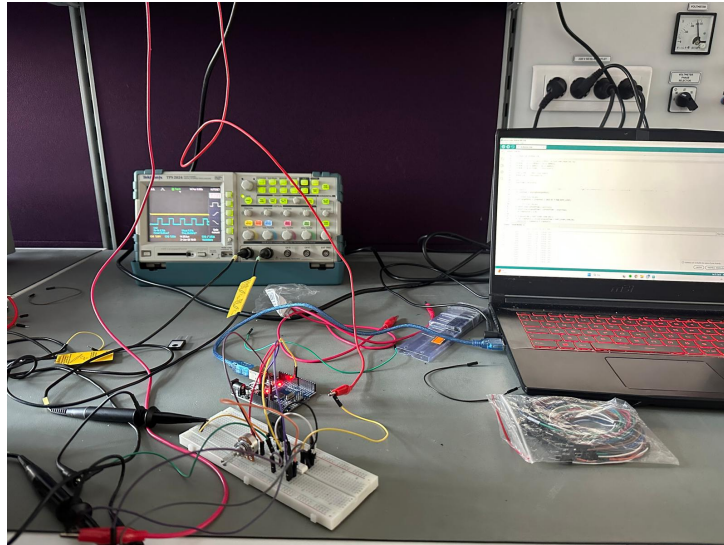


Figure 23: PWM Signal and Soft Start Test Setup

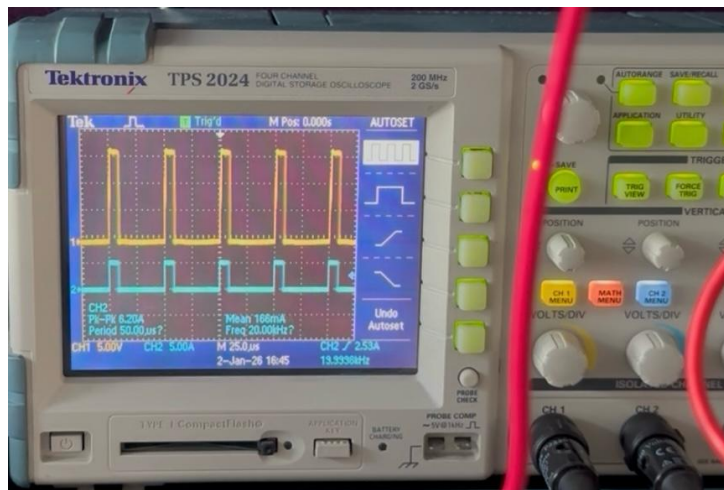


Figure 24: Initial Waveform During Soft Start

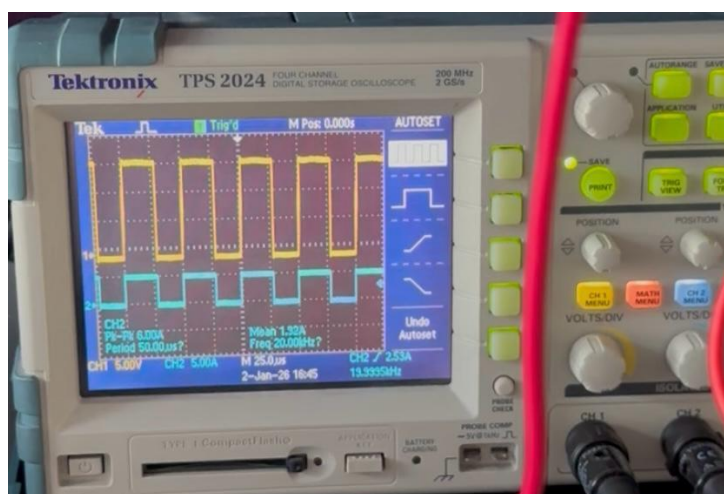


Figure 25: Final Waveform During Soft Start

After we observed the desired outcomes, we moved on to the implementation process. First, we soldered the DC-link capacitor, diode, MOSFET, and three-phase rectifier. We tested this setup to see whether we can get an output voltage with low ripple. We supplied the input from the variac, and we observed an almost constant (low ripple) capacitor voltage. We observed up to 60V in this stage due to safety considerations. We discharged the capacitor with a 68 ohm stone resistor and continued our implementation process.

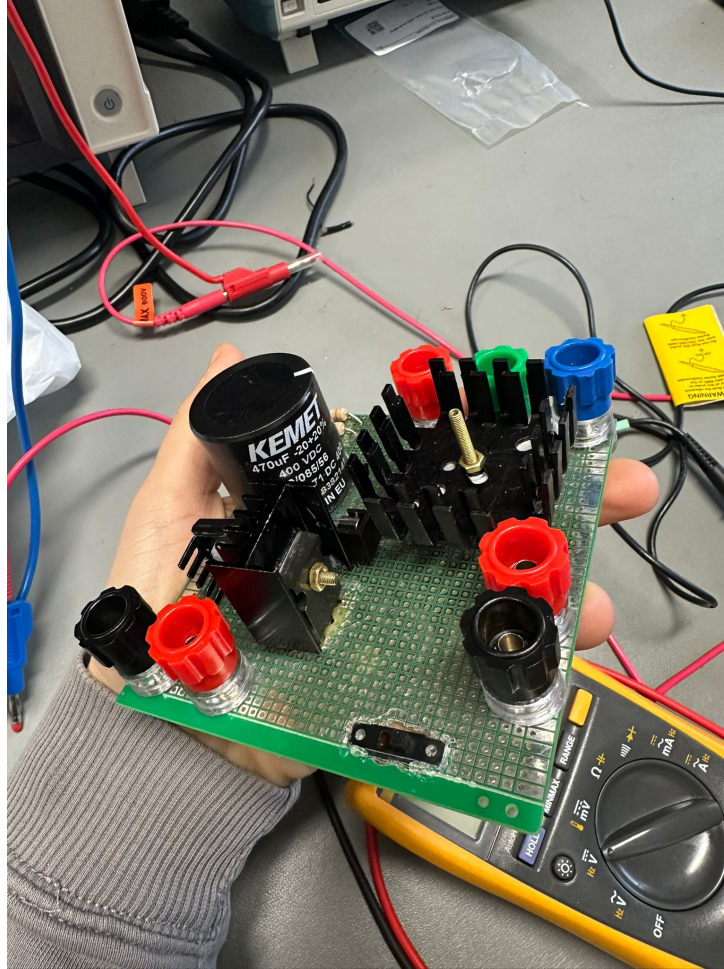


Figure 26: Rectifier and Output Voltage Test Setup

We decided to design the system in two layers, aiming for the compactness bonus. The top layer contains the power stage, which handles high current, and the gate driver. It includes the main switching and rectifying parts, such as the power MOSFET, rectifier, diode, and the main DC-link capacitor. Heatsinks and a fan were attached directly to the switching devices to prevent overheating during operation. This layer was built on a standard stripboard.

The bottom layer is used for low-voltage control. It holds the Arduino microcontroller and the voltage regulator. Small components, such as decoupling capacitors and gate resistors, were placed close to the main devices to reduce electrical noise.

The two layers were merged using spacers. This design provides good mechanical support and allows short, direct connections between the control circuit and the power switches. This design had some negative outcomes that will be mentioned in the demonstration part. After the implementation is completed (except layering), we tested the system with an R load under the guidance of our assistant and placed a cage for safety.

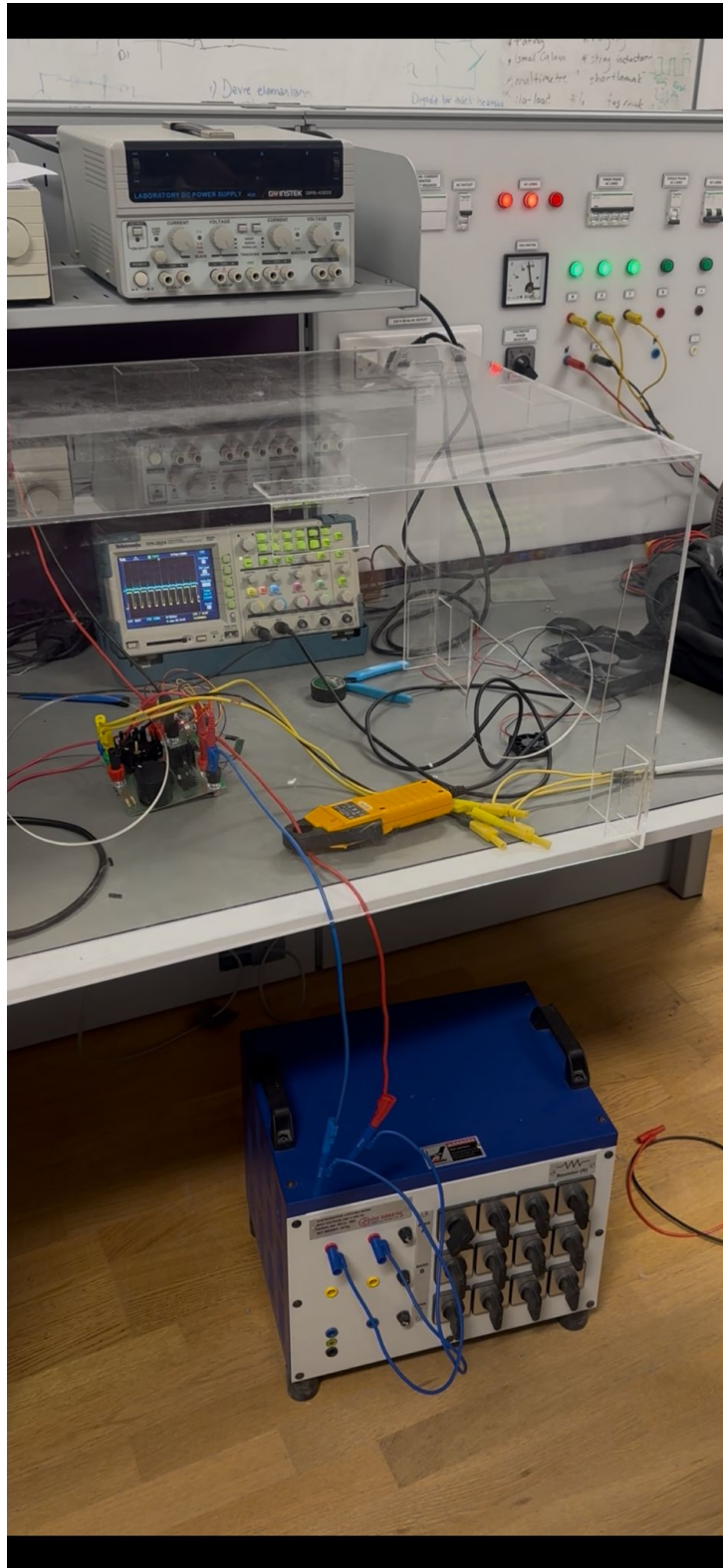


Figure 27: Full System Test Setup

Test results are as we wanted. MOSFET opened with soft start, supplying 15V DC to the system. After this test, considering the feedback of our assistant, we rearranged the soft start duration to 10 seconds to ensure safe opening and no inrush current. We observed a low ripple output voltage. We supplied high voltage from a variac (up to 180V) and observed the thermal state of the system. MOSFET and diode temperature went up to 47 degrees. Although that is acceptable, we decided to place a fan near these components, since we thought heatsinks may not be enough under high voltage and current, considering our thermal calculations.

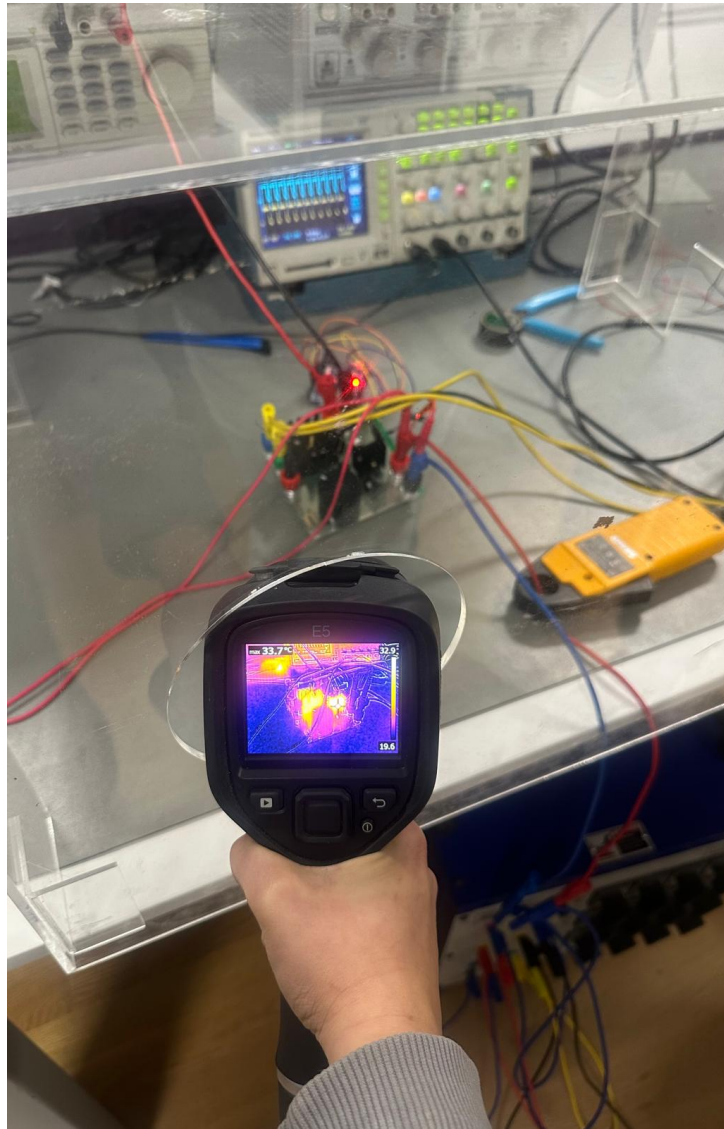


Figure 28: Thermal Measurement

Our fan is operating with 12V; however, we were supplying 15V. We connect a 10 ohm / 5W resistor to the fan to cause a voltage drop. After this, the layering process is completed, and we finished our implementation. Our final design is two layers, 10cmX10cm from the top view (10cmx12cm including potentiometer), and has 11 cm height.

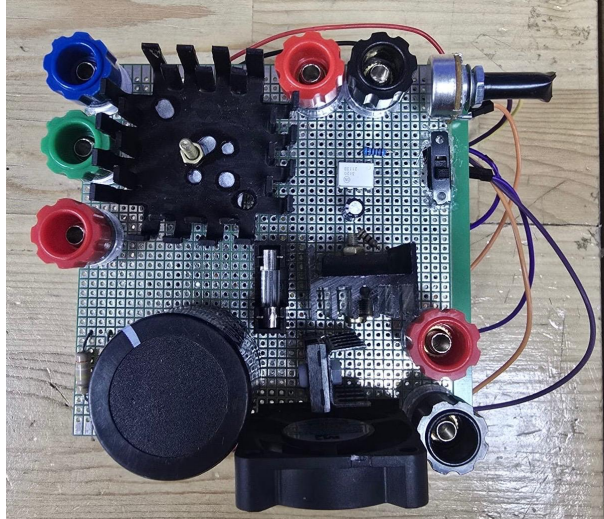


Figure 29: Top View of the Design

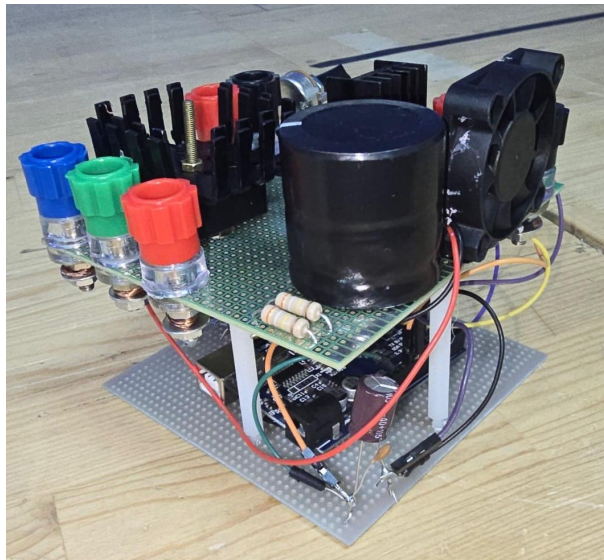


Figure 30: Corner View of the Design

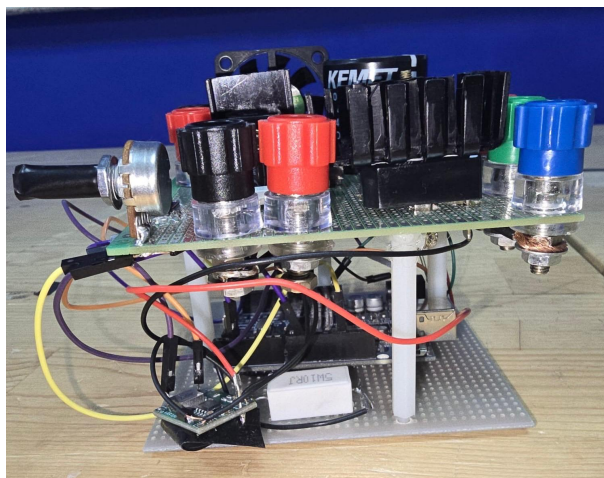


Figure 31: Side View of the Design

6.2 Demonstration Process

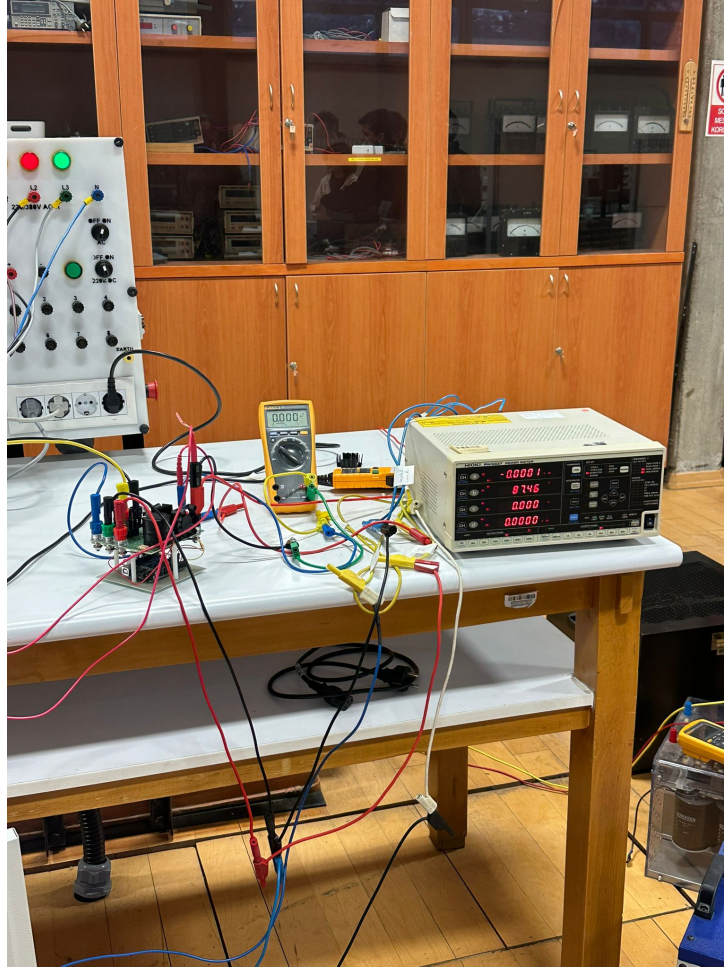


Figure 32: Demonstration Setup

Our initial demonstration was problematic. We tested successfully with R load, but connecting RL load caused some EMI problems. The MOSFET didn't open since we don't have isolated grounds, which increases the EMI problem. To overcome this, we connected capacitors (100 nF and 1 μ F in parallel) between the 5V and ground pins of the Arduino. However, this application didn't work.

The main problem was due to the placement of the Arduino. It was under MOSFET, and a high switching frequency creates EMI that affects the operation of Arduino. Finally, we managed to overcome this problem by moving Arduino aside. After this, our system worked well, drove the motor successfully from standstill up to 180V, and boiled the water in the kettle in 5 minutes.

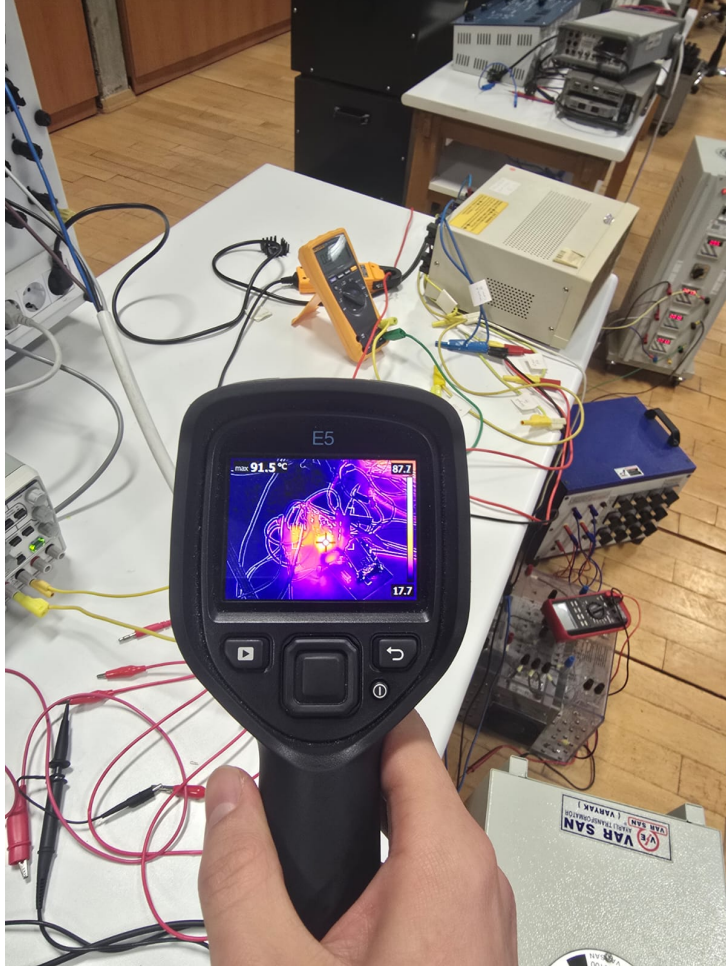


Figure 33: Thermal Measurement During Demonstration

During this final process, our MOSFET and diode heat up to 125 degrees, but we did not face any operational problems.

You can observe our demonstration results:

Table 1: Experimental Test Results

Test Case	$V_{in,rms}$ (V)	$P_{in,rms}$ (W)	$V_{o,avg}$ (V)	$P_{o,avg}$ (W)	Efficiency (η)
Motor Test (Low Voltage)	76.8	120	78	116	96.7%
Motor Test (Rated Voltage)	185	415	180	390	94.0%
Resistive Load (Kettle)	184	1290	178	1230	95.3%

The system showed strong overall performance during testing. It achieved high efficiency above 94% across all load conditions, with the highest power test using a kettle reaching an efficiency of 95.3% at a power level of 1.23 kW. The output voltage regulation was also satisfactory, as the output voltage closely followed the input during high duty-cycle operation, producing 180 V from a 185 V input. This confirms that the MOSFET and duty-cycle settings effectively applied the necessary output voltage. In addition, the system successfully handled a peak load of 1290 W without any thermal failure, validating the effectiveness of the cooling design using heatsinks and a fan.

7 Conclusion

In this project, we designed and built a high-voltage DC motor driver to operate a 1.6 kW separately excited DC motor. The system uses a three-phase rectifier followed by a buck converter to convert the three-phase AC supply into a controllable DC output of 180 V. We chose a low-side switching method because it simplifies the gate drive circuit and improves safety for the control electronics.

Before building the hardware, we verified the design using PSIM simulations. The simulation results showed stable operation under both no-load and full-load conditions. We observed that the soft-start algorithm effectively reduced the inrush current, helping to protect the motor and power devices. The simulations also confirmed that a 470 μF DC-link capacitor was sufficient to keep the voltage ripple within acceptable limits, even with the 300 Hz ripple from the rectified supply.

Component selection was an important part of the project. We selected the STW20NM60 MOSFET and the SKBPC5016 rectifier bridge because they provide enough voltage and current margins for safe operation. We used the FOD3120 gate driver due to its galvanic isolation, which protects the Arduino Uno from the high-voltage power stage. Thermal analysis showed that significant heat would be generated in the MOSFET and diodes, so we added an active cooling system.

During hardware testing, we encountered electromagnetic interference (EMI) issues when the Arduino was placed close to the high-frequency switching section. This caused incorrect controller behavior. We solved this problem by physically separating the control circuit from the power stage, which reduced EMI and noise.

Final experimental tests showed that the system worked as intended. We achieved the target output voltage of 180 V, and we optimized the soft-start time to 10 seconds for safe operation. Under full-load testing using a water-boiling setup, the active cooling system kept the semiconductor temperatures around 125 °C. This confirmed the accuracy of our thermal calculations and demonstrated that the system can operate safely and continuously.

8 References

1. onsemi, “FOD3120: High Noise Immunity, 2.5 A Output Current, Gate Drive Optocoupler,” Datasheet.
<https://www.onsemi.com/download/data-sheet/pdf/fod3120-d.pdf>
2. STMicroelectronics, “STB20NM60T4 / STP20NM60 / STW20NM60: N-Channel 600 V Power MOSFET,” Datasheet.
<https://www.st.com/resource/en/datasheet/stp20nm60.pdf>
3. Littelfuse (IXYS), “DSEI30-06A Fast Recovery Epitaxial Diode (FRED),” Datasheet.
<https://www.mouser.com/datasheet/2/205/DSEI30-06A-794218.pdf?srsltid=AfmB0orQax3d03C4yyVhtdAdv1zCVPnB4BWCnBfF7CmKIueATjCMYYuj>
4. *SKBPC5016 Three-Phase Bridge Rectifier*, Datasheet, Digi-Key Electronics.
5. Monolithic Power Systems (MPS), “MP1584: 3A, 1.5MHz, 28V Step-Down Converter,” Datasheet.
<https://content.instructables.com/FT8/0X4N/J9SVVWSX/FT80X4NJ9SVVWSX.pdf>

6. KEMET, “ALA7D Series Aluminum Electrolytic Capacitors (e.g., ALA7DA471DC400),” Datasheet.
<https://docs.rs-online.com/3c1d/0900766b81706f2e.pdf>
7. keysan.me

9 Appendix

9.1 Arduino Code

```
1 const int pwmPin = 9;          // FOD3120'ye giden sinyal
2 const int potPin = A0;         // Hiz
3 const int switchPin = 2;
4
5 const unsigned long SOFT_START_TIME_MS = 15000; // 15 Saniye
6 const float MAX_DUTY_LIMIT = 0.72;           // 180V Limiti
7
8 unsigned long startTime = 0;
9 bool motorRunning = false;
10
11 void setup() {
12     pinMode(pwmPin, OUTPUT);
13     pinMode(potPin, INPUT);
14
15     pinMode(switchPin, INPUT_PULLUP);
16
17     Serial.begin(9600);
18     Serial.println("Sistem Hazir (5kHz Modu). Switch bekleniyor...");
19
20     cli();
21     TCCR1A = 0; TCCR1B = 0;
22
23     TCCR1A |= (1 << COM1A1) | (1 << WGM11);
24     TCCR1B |= (1 << WGM13) | (1 << WGM12);
25
26     TCCR1B |= (1 << CS10);
27
28     // FREKANS: 16MHz / 5000Hz - 1 = 3199
29     ICR1 = 3199;
30
31     OCR1A = 0;
32     sei();
33 }
34
35 void loop() {
36     int inputVal = analogRead(potPin);
37
38     // Switch
39     bool switchState = digitalRead(switchPin);
40     bool switchAktif = (switchState == LOW);
41
42     if (switchAktif) {
43         if (!motorRunning) {
44             startTime = millis();
45             motorRunning = true;;
46         }
47
48         // Soft Start
49         unsigned long currentTime = millis();
50         unsigned long elapsedTime = currentTime - startTime;
51         float rampFactor = 1.0;
52
53         if (elapsedTime < SOFT_START_TIME_MS) {
54             float timeRatio = (float)elapsedTime / SOFT_START_TIME_MS;
```

```

55     rampFactor = timeRatio * timeRatio;
56 }
57
58 // Duty Hesabi
59 float targetDuty = (inputVal / 1023.0) * MAX_DUTY_LIMIT;
60 float effectiveDuty = targetDuty * rampFactor;
61
62 if (effectiveDuty > MAX_DUTY_LIMIT) effectiveDuty = MAX_DUTY_LIMIT;
63 if (effectiveDuty < 0.01) effectiveDuty = 0.0;
64
65 // PWM
66 OCR1A = (int)(effectiveDuty * 3199.0);
67
68 } else {
69     OCR1A = 0;
70     motorRunning = false;
71 }
72
73 delay(10);
74 }

```